

编译器设计实验-第二次实验

Haochen You

May 2026

1 实验任务（必做）

1.1 任务描述

用 C++ 实现一个词法分析程序 Scanner，支持两种运行模式：

模式 1：输入符号串个数 n ，然后输入 n 个空格分隔的符号串，程序对每个符号串进行词法识别，输出对应的 Token 类型（我这里也输出了词素）。

模式 2：输入一整行语句，程序对该语句进行词法分析，输出每个单词对应的 Token 类型和词素。

1.2 实验环境

操作系统：Windows 11

编译器：g++.exe (tdm64-1) 4.9.2

开发工具：Visual Studio Code

1.3 Token 种类设计

Token 类型	说明	示例
ID	标识符，字母开头，后跟字母或数字	a, num1, myVar
INT	关键字	int
FLOAT	关键字	float
VOID	关键字	void
IF	关键字	if
ELSE	关键字	else
WHILE	关键字	while
RETURN	关键字	return
INPUT	关键字	input
PRINT	关键字	print
NUM	整数，纯数字序列，支持正负	123, -5
FLO	浮点数，含小数、小数点开头、科学计数法	1.23, .45, 1e-3

Token 类型	说明	示例
ADD	加法运算符	+
SUB	减法运算符	-
MUL	乘法运算符	*
DIV	除法运算符	/
ASG	赋值运算符	=
AAS	加赋值运算符	+=
AAA	自增运算符	++
LT	小于	<
LE	小于等于	<=
EQ	等于	==
GT	大于	>
GE	大于等于	>=
NE	不等于	!=
AND	逻辑与	&&
OR	逻辑或	
NOT	逻辑非	!
LPA	左圆括号	(
RPA	右圆括号	)
LBK	左方括号	[
RBK	右方括号	]
LBR	左花括号	{
RBR	右花括号	}
CMA	逗号	,
SCO	分号	;

表 1: Token 种类表

如上表所示，一共设计了 36 种 Token，分为以下几类：标识符与关键字、数字常量、算术运算符、赋值运算符、关系运算符、逻辑运算符、括号与界符这七种。此外还有额外的一种 TOKEN_ERROR 来进行错误情况的处理。

1.4 DFA 构建

定义 $DFA(Q, \Sigma, \delta, q_0, F)$ 是我们要实现的 DFA。其中，输入集合包括：（字母、数字、运算符、括号与界符、小数点、指数符号、空白符）。下面的部分介绍状态集合的定义和初始状态，接受状态的定义：

1.4.1 DFA 状态集构建

状态	含义	接受状态
S_START	初始状态	否
S_ID	正在读标识符或关键字	是
S_NUM	正在读整数	是
S_FLO_DOT	读到小数点，从 S_NUM 转入	是
S_FLO_DOT_START	读到小数点，从 S_START 转入	否
S_FLO_DEC	正在读小数部分数字	是
S_FLO_E	读到指数符号 e/E	否
S_FLO_SIGN	读到指数后的正负号	否
S_FLO_EXP	正在读指数部分数字	是
S_ADD	读到 +，待确认后续	是
S_SUB	读到 -，待确认后续	是
S_ASG	读到 =，待确认后续	是
S_LT	读到 <，待确认后续	是
S_GT	读到 >，待确认后续	是
S_NOT	读到 !，待确认后续	是
S_AND	读到 &，待确认是否为 &&	否
S_OR	读到 ，待确认是否为	否

表 2: DFA 状态集合

可以看到，一共有 17 个状态，其中有一个初始状态 S_START，有 11 个接受状态。

1.4.2 DFA 转移函数

我们以列出转移路径的方法来展示 DFA 如何转移：

(1) 标识符与关键字（带自循环）

$$S_START \xrightarrow{\text{alpha}} S_ID \xrightarrow[\text{循环}]{\text{alpha|digit}} S_ID \xrightarrow{\text{其他}} \text{lookup_keyword()} / \text{TOKEN_ID}$$

(2) 运算符与逻辑符（按分支特征合并）

$$S_START \xrightarrow{<, >, !, =} \text{状态} \begin{cases} \xrightarrow{=} \text{双字符 (LE, GE, NE, EQ)} \\ \xrightarrow{\text{其他}} \text{单字符 (LT, GT, NOT, ASG)} \end{cases}$$

$$S_START \xrightarrow{+} S_ADD \begin{cases} \xrightarrow{=} \text{TOKEN_AAS (+)} \\ \xrightarrow{++} \text{TOKEN_AAA (++)} \\ \xrightarrow{\text{其他}} \text{TOKEN_ADD (+)} \end{cases}$$

$$S_START \xrightarrow{\&, |} S_AND, S_OR \xrightarrow{\&, |} \text{TOKEN_AND, TOKEN_OR}$$

(3) 数值解析主干（合并公共前缀 S_NUM）

$$S_START \xrightarrow{\text{digit}} S_NUM \begin{cases} \xrightarrow{\text{其他}} \text{TOKEN_NUM} & \text{(普通整数)} \\ \xrightarrow{\text{.}} S_FLO_DOT \xrightarrow{\text{digit}} S_FLO_DEC \xrightarrow{\text{其他}} \text{TOKEN_FLO} & \text{(小数)} \\ \xrightarrow{e|E} S_FLO_E \xrightarrow{+|-} S_FLO_SIGN \xrightarrow{\text{digit}} \dots \xrightarrow{\text{其他}} \text{TOKEN_FLO} & \text{(科学计数)} \end{cases}$$

(4) 纯单字符分隔符（集合映射）

$$S_START \xrightarrow{\{*, /, (,), [,], \{, \}, ,, ;\}} \text{直接返回对应 TOKEN}$$

1.5 代码实现逻辑

下面主要列出代码的主要的逻辑函数及数据定义：

定义两个枚举，TokenType 以及 DFAState，分别保存 Token 种类名和 DFA 状态名；

定义两个结构体，Token 和 Keyword，分别保存 Token 和关键字；

主函数判断是模式 1 还是模式 2，分别调用函数实现不同的模式逻辑，mode1() 调用 next_token()，其实现了 DFA 的逻辑以及转移。mode2() 调用 scan_line()，对一行字符串做词法分析，逐个打印 Token。

另外有两个函数 token_to_str 和 lookup_keyword，前者实现在关键字表中查找，找到返回对应类型，否则返回 TOKEN_ID，后者实现 Token 转字符串。

1.6 测试展示

先用简单的输入测试模式 1 和模式 2 的输出，结果如下：

```
(base) PS E:\2026_Junior_S2\编译器设计实验\第二次实验>
1
5
id if 485 841.6541 www
(ID, id)
(IF, if)
(NUM, 485)
(FLO, 841.6541)
(ID, www)
```

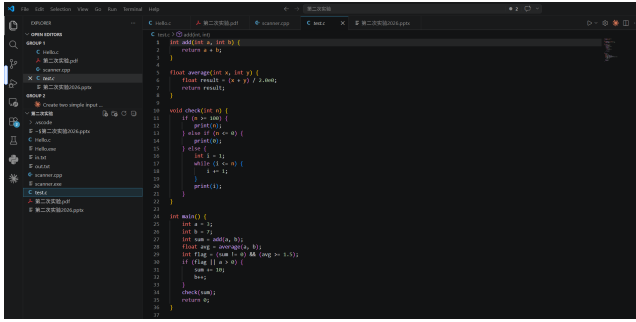
(a) Image 1

```
(base) PS E:\2026_Junior_S2\编译器设计实验\第二次实验> E:\2026_Junior_S2\编译器设计实验\第二次实验\scanner.exe
2
while(true) {int a=0;\}
(WHILE, while)
(ERROR, \)
(LPA, ()
(ID, true)
(ERROR, \)
(RPA, ))
(ERROR, \)
(LBR, {)
(INT, int)
(ID, a)
(ERROR, \)
(ASG, =)
(NUM, 0)
(ERROR, \)
(SCD, ;)
(ERROR, \)
(RBR, })
```

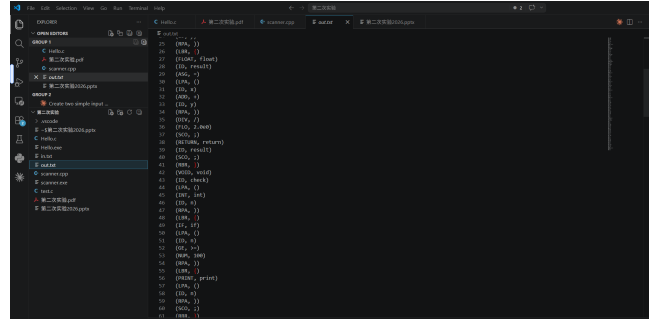
(b) Image 2

图 1: Test

下面我们用 freopen 实现文件的输入和输出，写一个 test.c，然后输出到 out.txt，测试如下：



(a) Image 1



(b) Image 2

图 2: Test

可以看到，Scanner 可以正常运行工作并输出文件，可以将词法分析的结果作为后续的输入。

2 实验任务（选做）

2.1 任务描述

实验中我们发现做完了词法分析之后，只能看到一行一行的输出，十分不直观。于是我们可以思考做一个前端文件，能够对词法分析的结果进行一个直接的展示（即能够实现代码高亮）。

主页面被分为左边和右边两个部分，左边是进行词法分析的源代码，右边是代码高亮后的结果。在右侧部分下面有逐 Token 的词法分析结果。

最后编写一个 html 文件，生成上面的功能。

2.2 实现思路

页面将 scanner.cpp 中的 DFA 词法分析逻辑用 JavaScript 重写了一遍。扫描函数 scan(src) 以字符串下标模拟 C 语言中的字符指针，维护与原程序相同的状态集合，逐字符进行状态转移，每识别出一个完整词素便记录其类型、原始字符串等信息，循环直至输入结束。

得到 Token 序列后，高亮渲染函数同步遍历原始源码与 Token 列表，将空白字符原样保留、词素部分用对应颜色的 标签包裹，从而能够生成带语法着色的 HTML。与此同时，Token 序列还被逐行写入右侧表格，呈现与实验二 out.txt 一致的输出，且能够在底部汇总各类别的数量。

2.3 展示

下面是实现后的前端页面展示：

词法分析器

编译原理设计实验二 - DFA-Based Lexical Scanner

SOURCE CODE 384 chars

开始扫描

HIGHLIGHTED SOURCE

```
int add(int a, int b) {
    return a + b;
}

float average(int x, int y) {
    float result = (x + y) / 2.0e0;
    return result;
}

void check(int n) {
    if (n >= 100) {
        print(n);
    } else if (n <= 0) {
        print(0);
    } else {
        int i = 1;
        while (i <= n) {
            i += 1;
        }
        print(i);
    }
}

int main() {
    int a = 3;
    int b = 7;
    int sum = add(a, b);
    float avg = average(a, b);
    int flag = (sum <= 0) && (avg >= 1.5);
    if (flag || a > 0) {
        sum += 10;
        b++;
    }
    check(sum);
    return 0;
}
```

```
1 int add(int a, int b) {
2     return a + b;
3 }
4
5 float average(int x, int y) {
6     float result = (x + y) / 2.0e0;
7     return result;
8 }
9
10 void check(int n) {
11     if (n >= 100) {
12         print(n);
13     } else if (n <= 0) {
14         print(0);
15     } else {
16         int i = 1;
17         while (i <= n) {
18             i += 1;
19         }
20         print(i);
21     }
22 }
23
24 int main() {
25     int a = 3;
26     int b = 7;
27     int sum = add(a, b);
28     float avg = average(a, b);
29     int flag = (sum <= 0) && (avg >= 1.5);
30     if (flag || a > 0) {
31         sum += 10;
32         b++;
33     }
34     check(sum);
35     return 0;
36 }
```

图 3: 前端 1

TOKEN STREAM			
#	TYPE	LEXEME	LINE
1	INT	int	1
2	ID	add	1
3	LPA	(	1
4	INT	int	1
5	ID	a	1
6	OPA	,	1
7	INT	int	1
8	ID	b	1
9	RPA	)	1
10	LSR	{	1
11	RETURN	return	2
12	ID	a	2
13	ADD	+	2
14	ID	b	2
15	SCO	;	2
16	RBR	}	3
17	FPLOAT	float	5
18	ID	average	5
19	LPA	(	5
20	INT	int	5
21	ID	x	5
22	OPA	,	5
23	INT	int	5
24	ID	y	5
25	RPA	)	5
26	LSR	{	5
27	FPLOAT	float	6
28	ID	result	6
29	ASS	=	6
30	LPA	(	6
31	ID	x	6
32	ADD	+	6
33	ID	y	6
34	RPA	)	6
35	DIV	/	6
36	FLO	2.0e0	6
37	SCO	;	6
38	RETURN	return	7
39	ID	result	7

图 4: 前端 2

3 个人总结

做这次实验之前，”词法分析”对我来说是一个挺抽象的概念。真正动手写完 DFA 之后才意识到，所谓”扫描”其实就是一个字一个字地往后看，靠着当前状态决定下一步怎么走，思路其实并不复杂，难的是把各种边界情况都想清楚，比如浮点数既可以从整数部分过来，也可以直接以小数点开头，科学计数法的符号位是可选的……这些细节稍不注意就会漏掉。选做部分把扫描器搬到网页上之后，反而对整个逻辑理解得更深了，因为 JavaScript 里没有指针，要自己用下标去模拟字符读取，让我把每一步的状态流转又捋了一遍。最后看到代码被颜色区分开来显示在页面上，还是挺有成就感的。